

Development Workflow, Testing & CI/CD

Complete Technical Guide — Lotwise Project

Table of Contents

- 1. What is Pre-Commit?
- 2. Code Quality Tools
- 3. Test Cases — Writing & Running
- 4. CI/CD Pipeline
- 5. How We Used It in Lotwise
- 6. Step-by-Step Installation & Usage

1. What is Pre-Commit?

Definition

Pre-commit is a robust framework that runs automated checks and quality assurance tests **BEFORE** you commit code to Git. It serves as the first line of defense in code maintenance.

Purpose

The primary objective is to intercept and catch code quality issues, syntax errors, and security vulnerabilities before they are permanently recorded in the repository history.

How It Works



Blocked! **Commit Proceeds ✓**
Files auto-fixed / manual review required

Benefits

- **Catch bugs early:** Identifies logical issues before they hit code review.
- **Enforce standards:** Automatically establishes uniform styling rules across the entire team.
- **Automatic resolution:** Self-corrects minor structural violations like trailing whitespaces and imports formatting.
- **Prevent debug leakage:** Restricts accidental commits of diagnostic logs (e.g., `console.log`, `print` statements).
- **Security enforcement:** Ensures credentials, passwords, and private tokens are never accidentally leaked.

Configuration File

Located at the repository root as `.pre-commit-config.yaml` :

```
repos:
- repo: https://github.com/charliermarsh/ruff-pre-commit
  rev: v0.1.6
  hooks:
    - id: ruff
    - id: ruff-format

- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.4.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer

- repo: https://github.com/pre-commit/mirrors-eslint
  rev: v8.44.0
  hooks:
    - id: eslint
```

2. Code Quality Tools

1. Ruff - Python Linter & Formatter

Ruff is an extremely fast Python linter and formatter that replaces multiple separate tools (like Black, Flake8, and isort). It parses code constraints and automatically aligns style rules.

Examples of Issues It Catches:

Wrong:

```
import os, sys # Multiple imports on one line
x=1 # No spaces around operator
def foo( ): # Spaces inside parentheses
    pass
```

Right:

```
import os
import sys

x = 1
def foo():
    pass
```

2. ESLint - JavaScript Linter

ESLint analyzes JavaScript source code to flag pattern anomalies, maintain code quality paradigms, and intercept runtime errors.

Wrong:

```
var x = 1; // Avoid var usage
if (x = 2) {} // Accidental assignment instead of comparison
console.log("debug"); // Leftover debug code
```

Right:

```
const x = 1;
if (x === 2) {}
```

3. Prettier - Code Formatter

An opinionated code formatter that standardizes style formats across JavaScript, JSON, Markdown, and YAML automatically.

```
// Before Prettier
const x=1;const y=2;
function foo(){return x+y;}

// After Prettier
const x = 1;
const y = 2;

function foo() {
```

```
    return x + y;
}
```

4. Pyupgrade - Python Syntax Modernizer

Upgrades older syntax conventions to modern, native implementations (e.g., changes old `"{0}".format(x)` string styles into modern clean f-strings `f"{x}"`).

3. Test Cases — Writing & Running

A test case is an isolated block of code written intentionally to verify that your target software feature operates correctly under predefined conditions.

Test Structure (Arrange-Act-Assert)

```
def test_create_tea_lot():
    # ARRANGE: Set up test state & objects
    seller = create_seller("Test Seller")
    mark = create_mark("Test Mark")

    # ACT: Execute the targeted block
    lot = create_tea_lot(seller=seller, mark=mark, packages=20)

    # ASSERT: Verify outcomes match parameters
    assert lot.seller == seller
    assert lot.packages == 20
    assert lot.status == "Offered"
```

Testing Types Employed

- **Unit Tests:** Isolates a single small processing function without external network/database dependencies.
- **Integration Tests:** Validates combined behavior of multiple layers working collectively (e.g., dynamic Excel uploads parsing cleanly into internal records).
- **Edge Case Tests:** Evaluates resilience against constraints, boundaries, or duplicate insertion attempts.

Running Tests in Frappe/Lotwise

```
# Run all test configurations for the app
bench --site lotwise.local run-tests --app lotwise

# Target a singular Doctype suite
bench --site lotwise.local run-tests --app lotwise --doctype "Tea Lot Import"
```

4. CI/CD Pipeline

Continuous Integration (CI) automatically verifies and runs test builds upon every individual commit upload to a central channel. **Continuous Deployment (CD)** executes hands-free delivery structures directly to live environments following comprehensive test clearance.

```
Code Push → Trigger Actions → Setup Env → Code Linting Checks → Test Suite Execution  
→ Automated Staging Deploy
```

GitHub Actions Configuration Example (tests.yml)

```
name: Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: 3.10
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install ruff eslint
      - name: Lint with ruff
        run: ruff check .
      - name: Run tests
        run: bench --site test.local run-tests --app lotwise
```

5. How We Used It in Lotwise

Every active component went through mandatory strict pipeline loops. This setup successfully established the following code coverage statistics within the repository environment:

- **Tea Lot Doctype:** 100% Code Coverage
- **Tea Evaluation Doctype:** 100% Code Coverage
- **Tea Lot Import Doctype:** 95% Code Coverage

6. Step-by-Step Installation & Usage

1. Setup Environments

```
pip install pre-commit
cd ~/lotwise_dev/apps/lotwise
pre-commit install
```

2. Run Checks Manually

```
pre-commit run --all-files
```

3. Continuous Automated Execution

Simply use standard Git processes. The engine automatically hooks verification parameters into `git commit` actions. If an infraction is detected, the operation stalls out and logs clear diagnostic output until resolved.

7. Summary: A to Z Workflow

Step	Action	Scope	Outcome / Control Point
1	Write Code	Local Environment	Developer creates or modifies feature logic.
2	<code>git commit</code>	Pre-Commit Hook	Intercepts the change before saving history.
3	Formatting & Linting	Automated Engine	Ruff and ESLint auto-fix patterns; blocks if issues remain.
4	<code>git push</code>	GitHub Remote	Triggers the cloud compilation pipeline.
5	Cloud CI Validation	GitHub Actions	Compiles code and runs full validation test suite.
6	Merge & Deploy	Production Control	Code base approved for release once tests pass successfully.